

How I used AI:

For the midterm check-in and “Prior work” section, I used ChatGPT in “Deep research” mode in order to find papers related to storage systems for AI-generated images, as well as to understand the current landscape in AIGI compression.

For the project presentation, I used Google Slides’ built-in slide “beautification” feature after manually typing in the content I wanted on the slides.

I used Codex for writing code for the CATC algorithm and for plotting results. I gave it the original paper for the algorithm, as well as the Github created by the algorithm’s authors (which needed a lot of additional code) to use as a framework for the code. I used the GPT-5.4 or 5.5 model with “Extra high” reasoning enabled. Initially I was using it on my laptop, and then installed it on the Chameleon testbed and connected it to my laptop’s Codex app by giving it the remote SSH connection. This allowed me to use the Codex app normally on my laptop while having it connected to more powerful compute resources on the Chameleon testbed.

To verify the correctness of my code, I read through the code, tried smaller test cases and examined the code output, and used separate Codex chats to verify the correctness (similar to GAN).

I didn’t use AI in writing the project report. In addition, all steps of the project were explicitly specified by me.

Reflection:

In the past, I’ve used ChatGPT for coding by describing the coding problem and copy-pasting the output. However, this class was my first time using a coding agent like Codex. Codex was able to run all of the code locally, and periodically ask for permission to do certain tasks like push to Github or install a necessary Python package. This increased coding efficiency substantially, as Codex was able to test the code on-the-fly and amend it right away. When I’ve used ChatGPT for coding in the past, it performed decently well on simpler coding tasks but the quality quickly deteriorated with increasing complexity. ChatGPT also tends to be “dumb” at fixing mistakes and often re-writes the entire code instead of just fixing small portions of it. I was very impressed with Codex, as it often made good assumptions that I didn’t specify. In contrast, in the past ChatGPT would only do exactly what you specify, like a purposefully obtuse genie.

I was also pleasantly surprised by Codex’s context window. It seems that Claude Code has issues with compressing its context window. With Codex, I didn’t notice any degradation in performance after receiving the “context window compression” notification.

Another consideration with coding agents is the rate limit. Here I was using Harvard’s ChatGPT Edu subscription for Codex. I didn’t end up hitting either the 5-hour or weekly rate limit, but I got pretty close to it. At one point I asked it to download a portion of the PKU-AIGI-500K dataset from Hugging Face, which quickly blew through a large chunk of the 5-hour rate limit. The takeaway from this is that tasks that take a long time but only take a few commands (like downloading a dataset or training a model) shouldn’t be delegated to a coding

agent, but they can help you form the command and then you run the command yourself to limit token usage.

The major downside of using coding agents heavily is that you can quickly get carried away and it becomes more difficult to have an in-depth understanding of how your code works. I tried to keep the coding “chunks” small and review the code as I went to retain an understanding of the code base.

Takeaways/suggestions:

- For long tasks that don't take a lot of commands (such as downloading datasets or training models), it's better to run the command yourself to avoid hitting your agent rate limit
- I haven't tried Claude Code or others but I had no issues with context window with Codex
- Try to maintain an understanding of the code as you build